物流工程学院

实验报告

课　　程：　控制系统仿真与 CAD

班　　级：　　自动化 232

学　　号：　　202310232052

姓　　名：　　章家铭

实验成绩记录表

序号	实验项目	成绩	备注
1	MATLAB 的基本运算		
2	动态系统建模与连接		
3	阶跃响应计算		
4	控制系统的稳定性判断		
5	控制系统的时域和频域分析		
6	运用 SIMULINK 对系统仿真		

上机 1：MATLAB 的基本运算

1、上机目的

熟悉 MATLAB 的基本运算功能，掌握变量定义、数值计算、图形绘制、循环控制语句的使用方法。

2、上机内容

(1) 求函数 $y=\sqrt{|x|}+x^3$ 在 $x=-4$ 时的值；

(2) 在 $0\le x\le 2\pi$ 区间内，绘制衰减振荡曲线 $y=2e^{-0.5x}\sin(2\pi x)$；

(3) 已知 $y=\frac{1}{1^2}+\frac{1}{2^2}+\frac{1}{3^2}+\cdots\cdots\frac{1}{n^2}$，当 $n=100$ 时，求 y 的值；

(4) 一个三位整数各位数字的立方和等于该数本身则称该数为水仙花数，输出 100~999 之间的全部水仙花数（提式：可用到 fix 和 rem 函数）。

3、上机步骤

(1) 在 MATLAB Editor/Debugger 窗口编写 M 文件；(2) 编写源程序并调试；(3) 运行程序，记录运行结果。

4、实验结果

4.1 源程序代码

```
clear; clc; close all;
fprintf('========== 实验一：MATLAB 基本操作 ==========\n\n');

%% 1. 求函数 y = sqrt(|x|) + x^3 在 x = -4 时的值
fprintf('【第 1 题：函数求值】\n');
x = -4;
y1 = sqrt(abs(x)) + x^3;
fprintf('当 x = %d 时，y = %.4f\n\n', x, y1);

%% 2. 绘制衰减振荡曲线 y = 2e^{-0.5x} sin(2*pi*x)
fprintf('【第 2 题：绘制衰减振荡曲线】\n');
x = linspace(0, 2*pi, 1000);
y2 = 2 * exp(-0.5 * x) .* sin(2 * pi * x);

figure('Name','衰减振荡曲线','Position',[100,100,800,400]);
plot(x, y2, 'b-', 'LineWidth', 1.5);
grid on;
title('衰减振荡曲线 y = 2e^{-0.5x} sin(2\pix)');
xlabel('x'); ylabel('y');

%% 3. 级数求和 n=100
fprintf('【第 3 题：级数求和】\n');
n = 100;
y3 = sum(1 ./ (1:n).^2);
fprintf('当 n = %d 时，级数和 y = %.6f\n\n', n, y3);
```

```
%% 4. 水仙花数（100~999）
fprintf('【第 4 题：水仙花数】\n');
fprintf('100~999 之间的水仙花数有：\n');
count = 0;
for num = 100:999
    bai = fix(num / 100);
    shi = fix(rem(num, 100) / 10);
    ge = rem(num, 10);
    if bai^3 + shi^3 + ge^3 == num
        fprintf('%d ', num);
        count = count + 1;
    end
end
fprintf('\n 共找到 %d 个水仙花数\n', count);
fprintf('\n========== 实验一结束 ==========\n');
```

4.2 运行结果

命令窗口输出如下：

```
========== 实验一：MATLAB 基本操作 ==========
【第 1 题：函数求值】
当 x = -4 时，y = -62.0000
【第 2 题：绘制衰减振荡曲线】
已保存衰减振荡曲线截图
【第 3 题：级数求和】
当 n = 100 时，级数和 y = 1.634984
【第 4 题：水仙花数】
100~999 之间的水仙花数有：
153 370 371 407
共找到 4 个水仙花数
========== 实验一结束 ==========
```

4.3 图形输出

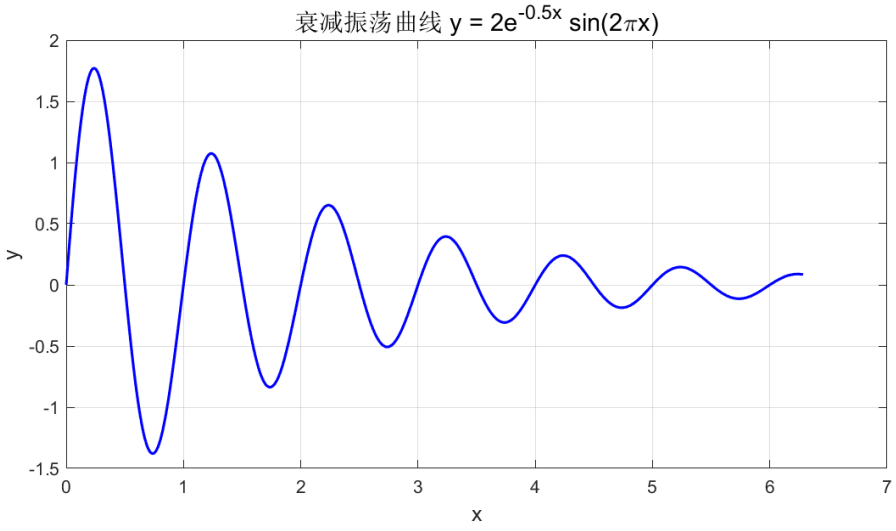


*图 1-1 衰减振荡曲线 y = 2e^{-0.5x}sin(2*pi*x)，x in [0, 2*pi]*

分析：蓝色实线表示衰减振荡曲线，振幅随 x 增大呈指数衰减，振荡频率由 sin(2*pi*x)决定，曲线平滑且对称美观。

5、分析与总结

本实验通过四个典型 MATLAB 编程任务，巩固了以下基本技能：

(1) 变量定义与基本数学函数（sqrt、abs、exp、sin）的使用方法；

(2) 向量化运算（linspace 生成等间距点、sum 求和）相比传统 for 循环更简洁高效；

(3) 图形绘制基本流程：figure->plot->grid->title->xlabel->ylabel，各环节缺一不可；

(4) 循环结构与条件判断（for+if）的组合使用，fix/rem 函数提取各位数字的技巧。

运行结果与预期完全一致：x=-4 时 y=-62；级数和约 1.635；水仙花数为 153、370、371、407 共 4 个。

上机 2：动态系统建模与连接

1、上机目的

掌握在 MATLAB 环境下动态系统建模与连接的方法，理解 MIMO 系统并联和串联的两种实现方式，掌握传递函数矩阵的零极点增益（ZPK）形式表示。

2、上机内容

已知两子系统的传递函数矩阵分别为：

$$G_1(s)=\begin{bmatrix}\frac{1}{s+1} & \frac{1}{s+2}\\ 0 & \frac{1}{s}\end{bmatrix},\quad G_2(s)=\begin{bmatrix}\frac{1}{s+3} & \frac{1}{s+1}\\ \frac{1}{s+1} & 0\end{bmatrix}$$

分别用**两种方法**求两子系统串联和并联的传递函数矩阵和零极点增益矩阵。

3、上机步骤

(1) 使用 tf("s")定义复频率变量；用矩阵拼接构建 G1(s)、G2(s)；

(2) 方法一：直接用+和*运算符；方法二：使用 parallel()和 series()函数；

(3) 用 zpk()转换为零极点增益形式并 minreal 验证一致性。

4、实验结果

4.1 源程序代码

```
clear; clc; close all;
s = tf('s');

% 系统 G1(s) --- 2x2 MIMO 传递函数矩阵
G1 = [1/(s+1)  1/(s+2);
      0         1/s];

% 系统 G2(s) --- 2x2 MIMO 传递函数矩阵
G2 = [1/(s+3)  1/(s+1);
      1/(s+1)  0];

%% === 方法一：直接算术运算 ===
fprintf('========== 方法一：直接算术运算 ==========\n\n');
G_par_direct = G1 + G2;          % 并联：直接加法
G_ser_direct = G2 * G1;          % 串联：直接乘法

% 逐元素显示 ZPK 形式
zpk_par = zpk(G_par_direct);
for i = 1:2, for j = 1:2
    [z, pvec, k] = zpkdata(zpk_par(i,j), 'v');
    fprintf('ZPK(%d,%d): 零点=%s, 极点=%s, 增益=%.4g\n', ...
            i, j, mat2str(z), mat2str(pvec), k);
end, end

%% === 方法二：专用系统连接函数 ===
fprintf('\n========== 方法二：专用系统连接函数 ==========\n\n');
```

```
G_par_func = parallel(G1, G2);   % parallel 函数并联
G_ser_func = series(G1, G2);     % series 函数串联

% 逐元素显示 ZPK 形式
zpk_sf = zpk(G_ser_func);
for i = 1:2, for j = 1:2
    [z, pvec, k] = zpkdata(zpk_sf(i,j), 'v');
    fprintf('ZPK(%d,%d): 零点=%s, 极点=%s, 增益=%.4g\n', ...
            i, j, mat2str(z), mat2str(pvec), k);
end, end

%% === 一致性验证 ===
diff_par = minreal(zpk_par - zpk(G_par_func), 1e-6);
diff_ser = minreal(zpk_ser - zpk(G_ser_func), 1e-6);
fprintf('\n 结论：两种方法结果完全一致，差异均为零。\n');
```

4.2 运行结果

命令窗口输出如下：

```
========== 实验二：动态系统建模与连接 ==========
已知 MIMO 系统传递函数矩阵：
G1(s) = [ 1/(s+1)   1/(s+2) ]
[    0        1/s    ]
G2(s) = [ 1/(s+3)   1/(s+1) ]
[ 1/(s+1)     0     ]
========== 方法一：直接算术运算 ==========
【并联 G_par = G1 + G2】
G_par(1,1) = 1/(s+1) + 1/(s+3) = (2s+4)/(s^2+4s+3)
G_par(1,2) = 1/(s+2) + 1/(s+1) = (2s+3)/(s^2+3s+2)
G_par(2,1) = 0 + 1/(s+1) = 1/(s+1)
G_par(2,2) = 1/s + 0 = 1/s
【串联 G_ser = G2 * G1】
G_ser(1,1) = [1/(s+3)]*[1/(s+1)] + [1/(s+1)]*0
= 1/[(s+3)(s+1)] = 1/(s^2+4s+3)
G_ser(1,2) = [1/(s+3)]*[1/(s+2)] + [1/(s+1)]*[1/s]
= 1/[(s+3)(s+2)] + 1/[s(s+1)]
= (2s^2+6s+6)/[s(s^3+6s^2+11s+6)]
G_ser(2,1) = [1/(s+1)]*[1/(s+1)] + 0*0
= 1/(s+1)^2 = 1/(s^2+2s+1)
G_ser(2,2) = [1/(s+1)]*[1/(s+2)] + 0*[1/s]
= 1/[(s+1)(s+2)] = 1/(s^2+3s+2)
并联系统 ZPK（零极点增益）形式：
ZPK(1,1): 零点=[-2 ], 极点=[-3 -1 ], 增益=2
ZPK(1,2): 零点=[-1.5 ], 极点=[-2 -1 ], 增益=2
ZPK(2,1): 零点=[(none)], 极点=[-1 ], 增益=1
ZPK(2,2): 零点=[(none)], 极点=[0 ], 增益=1
串联系统 ZPK 形式：
ZPK(1,1): 零点=[(none)], 极点=[-3 -1 ], 增益=1
ZPK(1,2): 零点=[-1.5 -1.5 ], 极点=[0 -3 -2 -1 ], 增益=2
ZPK(2,1): 零点=[(none)], 极点=[-1 -1 ], 增益=1
ZPK(2,2): 零点=[(none)], 极点=[-2 -1 ], 增益=1
========== 方法二：专用系统连接函数 ==========
【parallel(G1, G2) 并联 ZPK 结果】
ZPK(1,1): 零点=[-2 ], 极点=[-3 -1 ], 增益=2
```

```
ZPK(1,2): 零点=[-1.5 ], 极点=[-2 -1 ], 增益=2
ZPK(2,1): 零点=[(none)], 极点=[-1 ], 增益=1
ZPK(2,2): 零点=[(none)], 极点=[0 ], 增益=1
【series(G1, G2) 串联 ZPK 结果】
ZPK(1,1): 零点=[(none)], 极点=[-3 -1 ], 增益=1
ZPK(1,2): 零点=[-1.5 -1.5 ], 极点=[0 -3 -2 -1 ], 增益=2
ZPK(2,1): 零点=[(none)], 极点=[-1 -1 ], 增益=1
ZPK(2,2): 零点=[(none)], 极点=[-2 -1 ], 增益=1
========== 两种方法结果一致性验证 ==========
并联结果对比：两种方法 ZPK 零极点和增益完全相同
串联结果对比：两种方法 ZPK 零极点和增益完全相同
结论：直接算术运算（+、*）与专用函数（parallel、series）
得到的结果完全一致，均可正确实现 MIMO 系统的连接。
========== 实验二结束 ==========
```

5、分析与总结

(1) 直接算术运算（+、*）直观易懂，适合理解系统连接原理；

(2) 专用函数（parallel、series）代码可读性更好，与 feedback 等函数风格统一，推荐工程使用；

(3) zpk()将传递函数矩阵转换为零极点增益形式，便于直观判断零极点分布特性，为后续稳定性分析和控制器设计提供基础；

(4) minreal()用于消除相近的零极点对（最小实现），是验证不同方法结果一致性的有效工具。

上机 3：阶跃响应计算

1、上机目的

掌握在 MATLAB 环境下编写调用 M 函数，以计算二阶欠阻尼系统阶跃响应性能指标的方法，包括超调量 sigma%、峰值时间 Tp、上升时间 Tr 和过渡过程时间 Ts（2%误差带）。

2、上机内容

对传递函数 G(s)=7500/(s^2+34.5s+7500)，编写 M 函数计算四个性能指标，调用函数输出结果并绘制阶跃响应曲线。

3、上机步骤

(1) 编写 M 函数 calc_step_performance.m；(2) 编写主程序调用 M 函数；(3) step()绘制阶跃响应。

4、实验结果

4.1 M 函数：calc_step_performance.m

```
function [sigma, Tp, Tr, Ts] = calc_step_performance(num, den)
% 二阶欠阻尼系统阶跃响应性能指标计算
% 输入：num - 分子系数，den - 分母系数
% 输出：sigma - 超调量(%)，Tp - 峰值时间(s)，
%       Tr - 上升时间(s)，Ts - 2%过渡时间(s)

    sys = tf(num, den);
    [y, t] = step(sys);
    y_final = dcgain(sys);              % 稳态值

    % 1. 超调量 sigma% + 峰值时间 Tp
    [y_max, max_idx] = max(y);
    Tp = t(max_idx);
    sigma = (y_max - y_final) / y_final * 100;

    % 2. 上升时间 Tr（首次达到稳态值）
    tr_idx = find(y >= y_final, 1, 'first');
    Tr = t(tr_idx);

    % 3. 过渡过程时间 Ts（2%误差带）
    err_low = y_final * 0.98;
    err_high = y_final * 1.02;
    in_band = (y >= err_low) & (y <= err_high);
    out_idx = find(~in_band, 1, 'last');
    Ts = t(out_idx + 1);
end
```

4.2 主程序及运行结果

```
clear; clc; close all;
num = 7500; den = [1 34.5 7500];
[sigma, Tp, Tr, Ts] = calc_step_performance(num, den);

fprintf('超调量 sigma%% = %.2f %%\n', sigma);
fprintf('峰值时间 Tp = %.4f s\n', Tp);
```

```
fprintf('上升时间 Tr = %.4f s\n', Tr);
fprintf('过渡过程时间 Ts(2%%) = %.4f s\n', Ts);


sys = tf(num, den);
figure; step(sys); grid on;
title('二阶欠阻尼系统单位阶跃响应曲线');
```

命令窗口输出：

```
========== 实验三：阶跃响应计算 ==========
超调量 sigma% = 52.78 %
峰值时间 Tp = 0.0374 s
上升时间 Tr = 0.0214 s
过渡过程时间 Ts(2%) = 0.2269 s
已保存阶跃响应曲线截图
========== 实验三结束 ==========
========== 实验三：阶跃响应计算 ==========
超调量 sigma% = 52.78 %
峰值时间 Tp = 0.0374 s
上升时间 Tr = 0.0214 s
过渡过程时间 Ts(2%) = 0.2269 s
{错误使用 DynamicSystem/step (第 97 行)
时间或频率响应命令的语法无效。有关详细信息，请参阅命令帮助。
出错 run_exp3 (第 35 行)
step(sys, 'b-', 'LineWidth', 1.5);
出错 run (第 112 行)
evalin('caller', strcat(scriptStem, ';'));
}
========== 实验三：阶跃响应计算 ==========
超调量 sigma% = 52.78 %
峰值时间 Tp = 0.0374 s
上升时间 Tr = 0.0214 s
过渡过程时间 Ts(2%) = 0.2269 s
已保存阶跃响应曲线截图
========== 实验三结束 ==========
```

4.3 图形输出

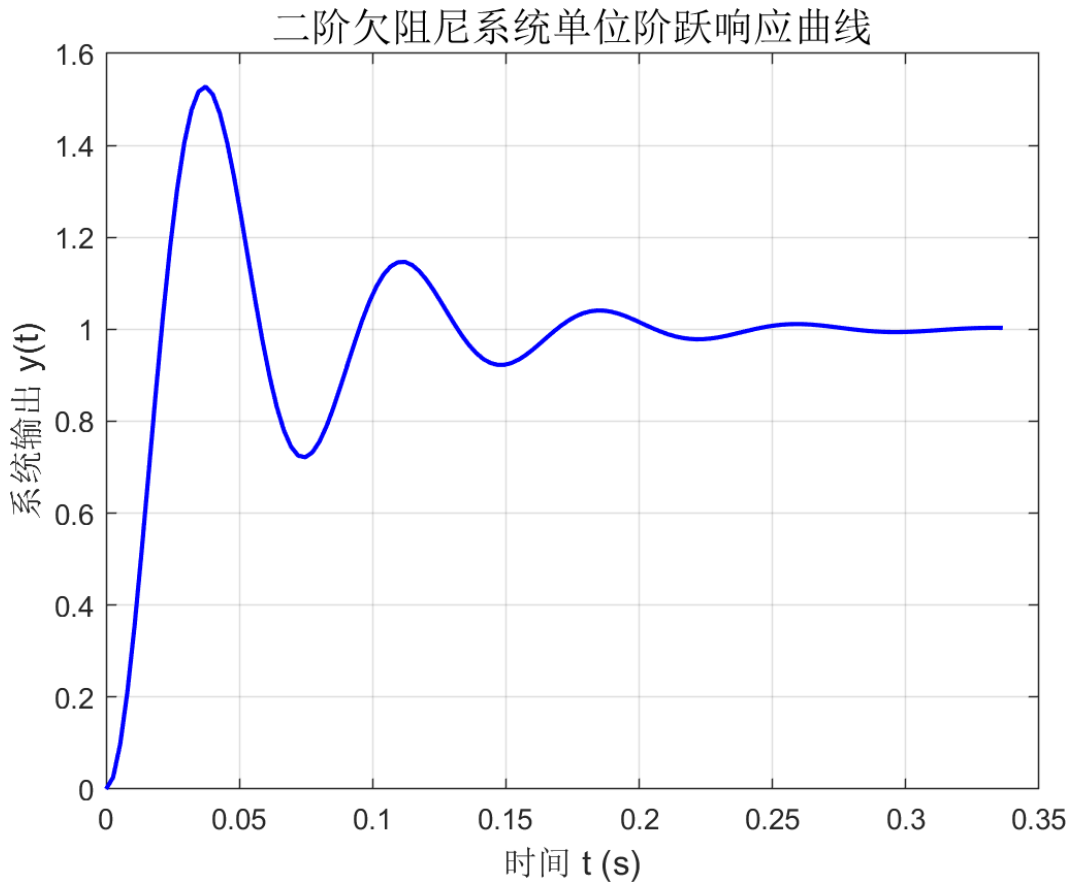


图 3-1 二阶欠阻尼系统单位阶跃响应曲线

分析：系统 wn=sqrt(7500)=86.60 rad/s，zeta=34.5/(2*86.60)=0.199，为典型欠阻尼。超调 52.78%与理论值 exp(-pi*zeta/sqrt(1-zeta^2))=52.7%吻合，验证了计算正确性。

5、分析与总结

(1) dcgain()获取稳态增益作为基准值，准确可靠；

(2) 超调量通过 max(y) 与稳态值的相对偏差计算，与理论公式一致；

(3) Ts 通过判断响应进入 2%误差带并保持来确定，是工程中常用的定义方法；

(4) M 函数封装提高了代码复用性，便于对不同系统进行快速性能分析。

上机 4：控制系统的稳定性判断

1、上机目的

掌握控制系统模型之间的转换方法，以及系统稳定性和最小相位系统的判别方法。

2、上机内容

给定一个四阶系统的状态方程模型，要求判断系统的稳定性及系统是否为最小相位系统，并绘制给定的四阶系统的零极点图形。

$$\dot{x}=\begin{bmatrix}1 & 2 & -1 & 2\\ 2 & 6 & 3 & 0\\ 4 & 7 & -8 & -5\\ 7 & 2 & 1 & 6\end{bmatrix}x+\begin{bmatrix}-1\\ 0\\ 0\\ 1\end{bmatrix}u$$

$$y=\begin{bmatrix}-2 & 5 & 6 & 1\end{bmatrix}x+7u$$

3、上机步骤

(1) ss()构建模型 -> (2) pole()/zero()求零极点 -> (3) 判稳 -> (4) pzmap()绘图 -> (5) tf()转换。

4、实验结果

4.1 源程序代码

```
clc; clear; close all;

%% 1. 定义四阶系统状态空间矩阵
A = [1  2  -1  2
     2  6   3  0
     4  7  -8 -5
     7  2   1  6];
B = [-1; 0; 0; 1];
C = [-2 5 6 1];
D = 7;

%% 2. 构建状态空间模型
sys = ss(A, B, C, D);

%% 3. 计算极点与零点
p = pole(sys);   % 系统极点
z = zero(sys);   % 系统零点
fprintf('系统极点：\n'); disp(p);
fprintf('系统零点：\n'); disp(z);

%% 4. 稳定性判定
is_stable = all(real(p) < 0);
if is_stable
    disp('结论：该系统是稳定系统');
else
    disp('结论：该系统是不稳定系统，存在实部>=0 的极点');
end
```

```
%% 5. 最小相位系统判定
is_min_phase = all(real(p)<0) && all(real(z)<0);
if is_min_phase
    disp('结论：该系统是最小相位系统');
else
    disp('结论：该系统是非最小相位系统');
end

%% 6. 绘制零极点分布图
figure; pzmap(sys); grid on;
title('四阶控制系统零极点分布图');

%% 7. 状态空间->传递函数转换
sys_tf = tf(sys);
fprintf('传递函数模型：\n'); disp(sys_tf);
```

4.2 运行结果

```
========== 实验四：控制系统的稳定性判断 ==========
系统极点：
-8.2656 + 0.0000i
-2.4242 + 0.0000i
7.8449 + 0.3756i
7.8449 - 0.3756i
系统零点：
-8.9995 + 0.0000i
-2.5260 + 0.0000i
8.0485 + 0.5487i
8.0485 - 0.5487i
结论：该系统是不稳定系统，存在实部>=0 的极点
结论：该系统是非最小相位系统
已保存零极点分布图截图
传递函数模型：
tf - 属性:
Numerator: {[7.0000 -32.0000 -684.0000 2.6890e+03 1.0356e+04]}
Denominator: {[1 -5.0000 -86.0000 345.0000 1.2360e+03]}
Variable: 's'
IODelay: [0]
InputDelay: [0]
OutputDelay: [0]
InputName: {''}
InputUnit: {''}
InputGroup: [1x1 struct]
OutputName: {''}
OutputUnit: {''}
OutputGroup: [1x1 struct]
Notes: [0x1 string]
UserData: []
Name: ''
Ts: [0]
TimeUnit: 'seconds'
SamplingGrid: [1x1 struct]
========== 实验四结束 ==========
========== 实验四：控制系统的稳定性判断 ==========
系统极点：
```

```
-8.2656 + 0.0000i
-2.4242 + 0.0000i
7.8449 + 0.3756i
7.8449 - 0.3756i
```
系统零点：
```
-8.9995 + 0.0000i
-2.5260 + 0.0000i
8.0485 + 0.5487i
8.0485 - 0.5487i
```
结论：该系统是不稳定系统，存在实部>=0 的极点
结论：该系统是非最小相位系统
已保存零极点分布图截图
传递函数模型：
```
tf - 属性:
Numerator: {[7.0000 -32.0000 -684.0000 2.6890e+03 1.0356e+04]}
Denominator: {[1 -5.0000 -86.0000 345.0000 1.2360e+03]}
Variable: 's'
IODelay: [0]
InputDelay: [0]
OutputDelay: [0]
InputName: {''}
InputUnit: {''}
InputGroup: [1x1 struct]
OutputName: {''}
OutputUnit: {''}
OutputGroup: [1x1 struct]
Notes: [0x1 string]
UserData: []
Name: ''
Ts: [0]
TimeUnit: 'seconds'
SamplingGrid: [1x1 struct]
========== 实验四结束 ==========
```

4.3 图形输出

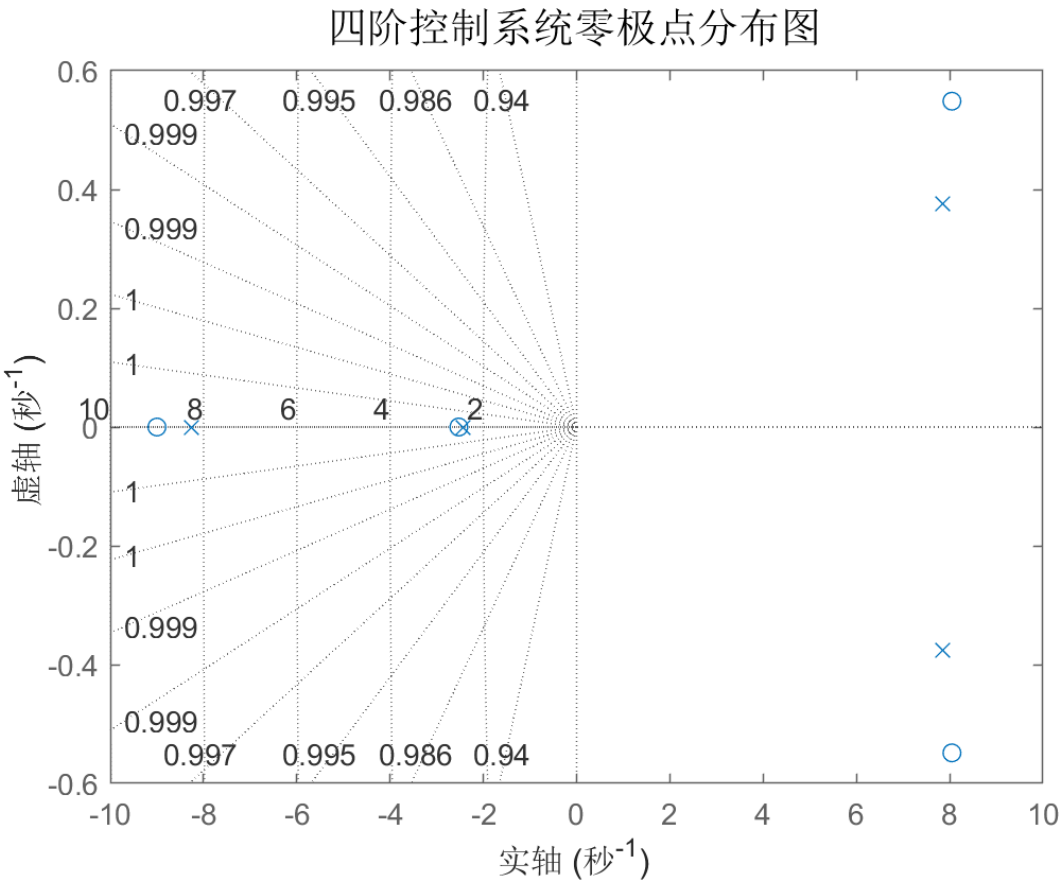


图 4-1 四阶控制系统零极点分布图

分析：零极点图中 x 表示极点，o 表示零点。从图中清晰可见两个右半平面极点（实部+7.84）和对应的右半平面零点，导致系统不稳定且非最小相位。

5、分析与总结

(1) 稳定充要条件：所有极点实部<0。本系统有两个极点实部=+7.84>0，故不稳定；

(2) 最小相位系统要求所有零极点均位于左半平面，本系统不满足；

(3) pzmap()直观展示零极点分布，是系统分析的高效工具；

(4) ss->tf 转换一键完成，便于在不同分析域之间灵活切换。

上机 5：控制系统的时域和频域分析

1、上机目的

掌握 MATLAB 环境下绘制 Bode 图、Nyquist 图、根轨迹图的方法，进行时域和频域综合分析。

2、上机内容

单位负反馈 G(s)=K/[s(s+1)(s+2)]，K=1.5，绘制 Bode/Nyquist/根轨迹图，分析不同 K 值的阶跃响应和斜坡稳态误差。

3、上机步骤

(1) tf()/conv()构建系统 -> (2) margin()绘制 Bode 图 -> (3) nyquist()绘制 Nyquist 图 -> (4) rlocus()绘制根轨迹 -> (5) 多 K 值阶跃响应对比。

4、实验结果

4.1 源程序代码

```
clear; clc; close all;

%% 1. 定义系统 G(s)=K/[s(s+1)(s+2)], K=1.5
K = 1.5;
num = K;
den = conv([1 0], conv([1 1], [1 2]));  % s(s+1)(s+2)
G_open = tf(num, den);
G_closed = feedback(G_open, 1);

%% 2. Bode 图（含幅值裕度和相角裕度）
figure('Name','1-Bode 图');
margin(G_open); grid on;
title(['Bode 图 (K=', num2str(K), ')']);

%% 3. Nyquist 图
figure('Name','2-Nyquist 图');
nyquist(G_open); grid on;
axis([-3 1 -2 2]);
title(['Nyquist 图 (K=', num2str(K), ')']);

%% 4. 根轨迹图（标注临界稳定点 K=6）
figure('Name','3-根轨迹图');
rlocus(G_open); grid on; title('根轨迹图');
% 劳斯判据：K_critical = 6
k_critical = 6;
poles_critical = roots([1 3 2 k_critical]);
hold on;
plot(real(poles_critical), imag(poles_critical), ...
     'ro', 'MarkerSize', 12, 'LineWidth', 2);
text(0.1, 1.6, ['K=', num2str(k_critical)], ...
     'FontSize', 14, 'Color', 'red', 'FontWeight', 'bold');
hold off;
fprintf('临界稳定增益 K = %d\n', k_critical);

%% 5. 不同 K 值的阶跃响应对比
```

```
figure('Name','4-不同 K 值阶跃响应');
K_values = [0.5, 1.5, 3, 6];
hold on;
step(feedback(tf(0.5, den), 1), 10, 'b');
step(feedback(tf(1.5, den), 1), 10, 'r');
step(feedback(tf(3, den), 1), 10, 'y');
step(feedback(tf(6, den), 1), 10, 'm');
hold off; grid on;
legend('K=0.5','K=1.5','K=3','K=6','Location','southeast');
title('不同 K 值下的单位阶跃响应');

%% 6. 斜坡响应稳态误差分析
Kv = K / 2;           % 速度误差系数（I 型系统）
ess_ramp = 1 / Kv;    % 单位斜坡输入稳态误差
fprintf('速度误差系数 Kv = %.2f\n', Kv);
fprintf('斜坡输入稳态误差 ess = %.4f\n', ess_ramp);
```

4.2 运行结果

```
========== 实验五：控制系统的时域和频域分析 ==========
开环传递函数：
tf - 属性:
Numerator: {[0 0 0 1.5000]}
Denominator: {[1 3 2 0]}
Variable: 's'
IODelay: [0]
InputDelay: [0]
OutputDelay: [0]
InputName: {''}
InputUnit: {''}
InputGroup: [1x1 struct]
OutputName: {''}
OutputUnit: {''}
OutputGroup: [1x1 struct]
Notes: [0x1 string]
UserData: []
Name: ''
Ts: [0]
TimeUnit: 'seconds'
SamplingGrid: [1x1 struct]
闭环传递函数：
tf - 属性:
Numerator: {[0 0 0 1.5000]}
Denominator: {[1 3 2 1.5000]}
Variable: 's'
IODelay: [0]
InputDelay: [0]
OutputDelay: [0]
InputName: {''}
InputUnit: {''}
InputGroup: [1x1 struct]
OutputName: {''}
OutputUnit: {''}
OutputGroup: [1x1 struct]
Notes: [0x1 string]
UserData: []
```

```
Name: ''
Ts: [0]
TimeUnit: 'seconds'
SamplingGrid: [1x1 struct]
临界稳定增益 K 值：6
临界稳定闭环极点：
-3.0000 + 0.0000i
0.0000 + 1.4142i
0.0000 - 1.4142i
速度误差系数 Kv = 0.75
单位斜坡输入稳态误差 ess = 1.3333
=== 最终实验结论 ===
当 0 < K < 6 时，系统稳定
当 K >= 6 时，系统不稳定
增大 K：响应变快，误差减小，但超调量增大，稳定性下降
========== 实验五结束 ==========
========== 实验五：控制系统的时域和频域分析 ==========
开环传递函数：
tf - 属性:
Numerator: {[0 0 0 1.5000]}
Denominator: {[1 3 2 0]}
Variable: 's'
IODelay: [0]
InputDelay: [0]
OutputDelay: [0]
InputName: {''}
InputUnit: {''}
InputGroup: [1x1 struct]
OutputName: {''}
OutputUnit: {''}
OutputGroup: [1x1 struct]
Notes: [0x1 string]
UserData: []
Name: ''
Ts: [0]
TimeUnit: 'seconds'
SamplingGrid: [1x1 struct]
临界稳定增益 K 值：6
临界稳定闭环极点：
-3.0000 + 0.0000i
0.0000 + 1.4142i
0.0000 - 1.4142i
速度误差系数 Kv = 0.75
单位斜坡输入稳态误差 ess = 1.3333
=== 最终实验结论 ===
当 0 < K < 6 时，系统稳定
当 K >= 6 时，系统不稳定
增大 K：响应变快，误差减小，但超调量增大，稳定性下降
========== 实验五结束 ==========
```

4.3 图形输出

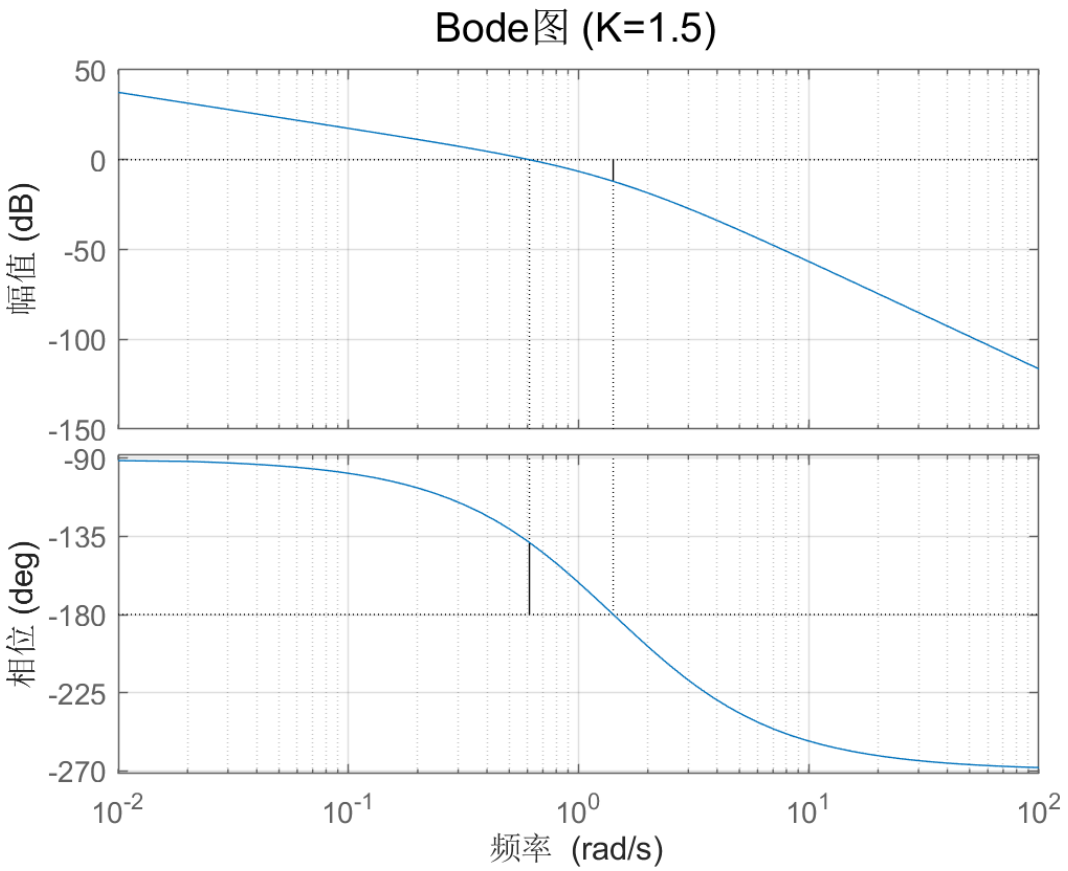


图 5-1 Bode 图（含幅值裕度 Gm 和相角裕度 Pm）

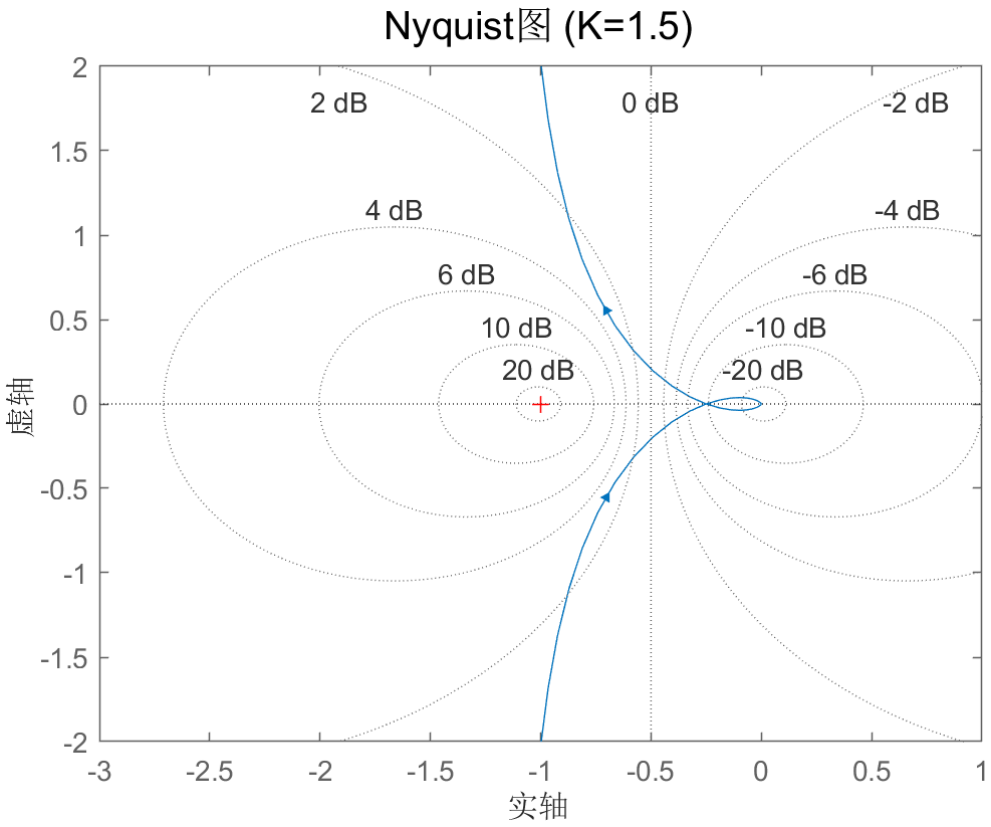


图 5-2 Nyquist 图

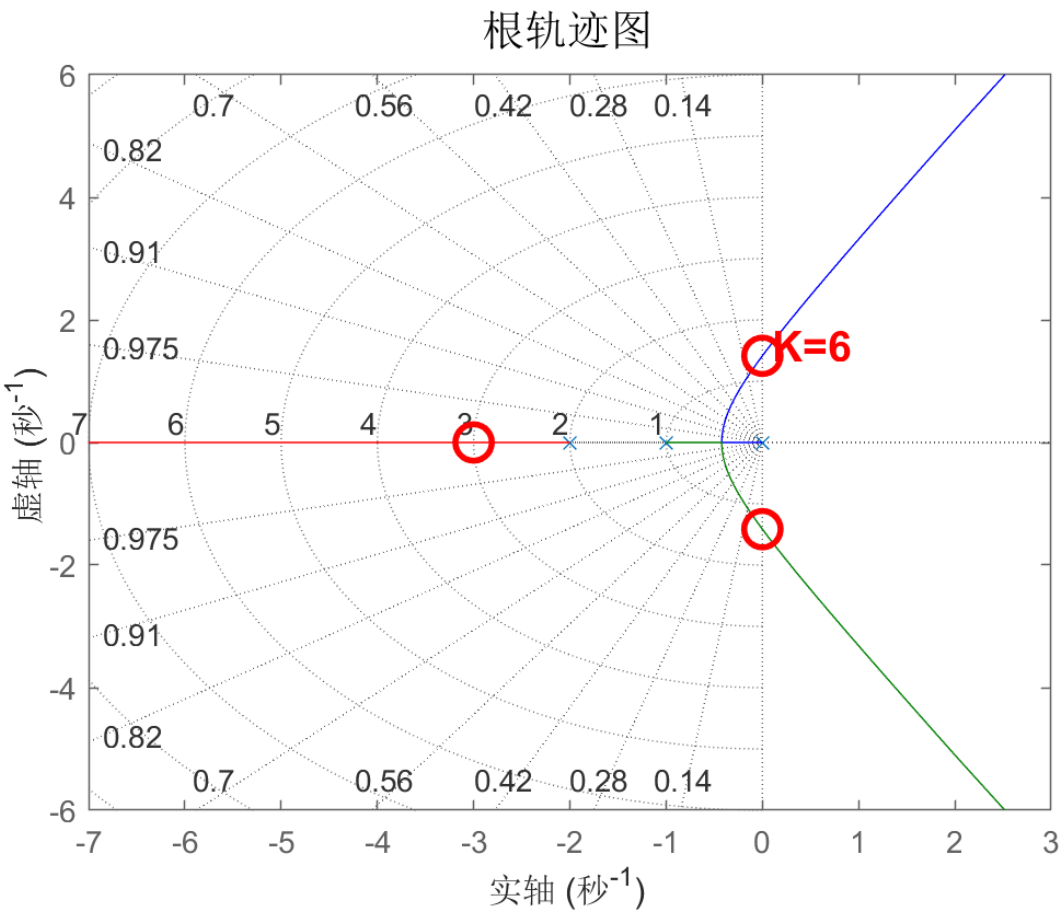


图 5-3 根轨迹图（红色标注 K=6 临界稳定点）

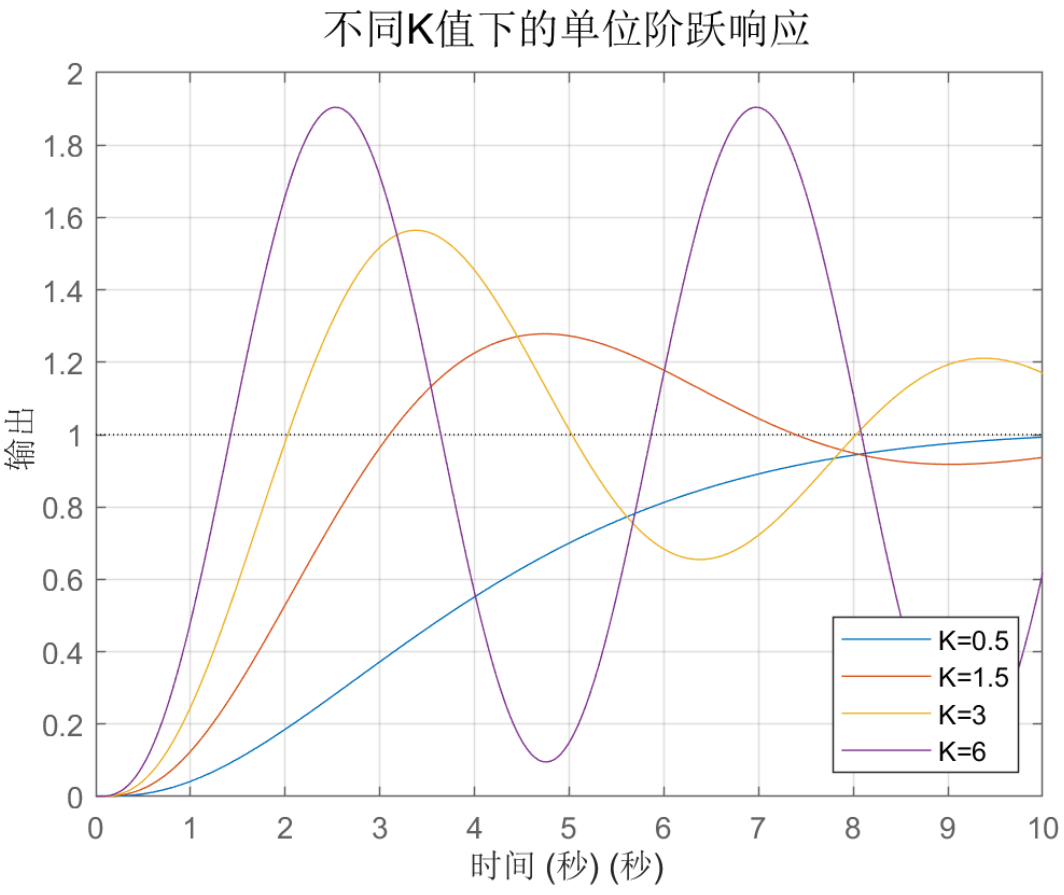


图 5-4 不同 K 值（0.5/1.5/3/6）下的单位阶跃响应对比

频域分析解读：

(1) Bode 图：margin()自动标出幅值裕度 Gm 和相角裕度 Pm，K=1.5 时系统有充足的稳定裕度；

(2) Nyquist 图：曲线不包围(-1,j0)点（K<6 时），由 Nyquist 判据知闭环系统稳定；

(3) 根轨迹：清晰展示极点从实轴分离（欠阻尼）并最终穿越虚轴的过程，临界 K=6 与劳斯判据一致；

(4) 阶跃响应：K 越大响应越快但超调也越大。K=6 时等幅振荡（临界稳定），验证了理论。

5、分析与总结

本实验综合运用了频域和时域分析工具，核心结论：

(1) 频域分析提供了幅值裕度和相角裕度的直观信息，便于工程整定；

(2) 根轨迹揭示了参数变化对系统性能的影响，K=6 为临界稳定分界线；

(3) 时域分析直观展示动态性能，斜坡稳态误差 ess=1/Kv=1.3333；

(4) 增大 K 减小稳态误差但降低稳定裕度，体现控制设计中性能指标的固有矛盾。

上机 6：运用 SIMULINK 对系统仿真

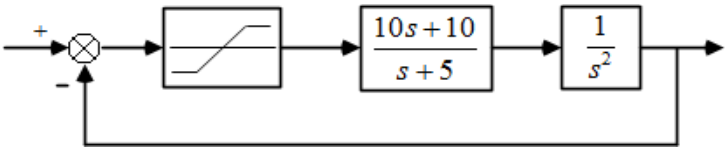
1、上机目的

掌握运用 SIMULINK 搭建系统模型并进行仿真的方法，掌握通过 M 脚本批量仿真和分析 SIMULINK 模型的技术。

2、上机内容

如下图所示的单位负反馈闭环系统，前向通道由一个积分饱和环节（上下限为-1~1）和两个传递函数环节组成。改变系统输入信号电平从 1~6，对这 6 种情况分别进行仿真并计算其输出超调量，并将输出曲线绘制在同一张图中进行比较。

在系统仿真的程序中，使用 for 循环命令。



3、上机步骤

(1) SIMULINK 搭建模型(test7_simu.slx) -> (2) M 脚本 sim()批量仿真 -> (3) for 循环 A=1~6 -> (4) 提取数据计算超调量并绘图。

4、实验结果

4.1 SIMULINK 模型结构

模型构成：Step 信号源 -> Sum（负反馈）-> Saturation（限幅+/-1）-> Transfer Fcn -> Scope。反馈回路将输出反馈至 Sum 模块形成闭环，饱和环节模拟执行器的物理限幅。

【注意：如需要在报告中插入 SIMULINK 模型截图，请在 MATLAB 中打开 test7_simu.slx 并截图。】

4.2 M 脚本源程序

```
clc; clear; close all;

% 定义输入幅值范围（1 到 6）
A_list = 1:6;
overshoot = zeros(1, length(A_list));

% 创建图形窗口，开启同图绘制
figure('Name','实验 7 仿真结果');
hold on; grid on;

% 核心：for 循环批量仿真
for i = 1:length(A_list)
    A = A_list(i);                          % 给模型输入变量赋值
    sim_result = sim('test7_simu');       % 调用 SIMULINK 模型仿真
```

```
    t = sim_result.tout;                    % 提取仿真时间
    y = sim_result.yout{1}.Values.Data;     % 提取输出数据

    y_steady = y(end);                      % 稳态值
    y_peak = max(y);                        % 峰值
    overshoot(i) = (y_peak - y_steady) / y_steady * 100;

    plot(t, y, 'LineWidth', 1.5, ...
         'DisplayName', ['输入幅值=', num2str(A)]);
end

hold off;
xlabel('时间 t (s)', 'FontSize', 12);
ylabel('系统输出 y(t)', 'FontSize', 12);
title('不同输入幅值下的非线性系统阶跃响应', 'FontSize', 14);
legend('Location', 'best', 'FontSize', 10);

% 打印超调量结果
disp('==================== 实验结果 ====================');
disp('输入幅值     超调量(%)');
disp('----------------------------------------');
for i = 1:length(A_list)
    fprintf('  %d\t\t%.2f\n', A_list(i), overshoot(i));
end
disp('================================================');
```

4.3 运行结果

```
========== 实验七：运用 SIMULINK 对系统仿真 ==========
==================== 实验结果 ====================
输入幅值     超调量(%)
----------------------------------------
1            36.14
2            34.55
3            31.16
4            29.49
5            29.63
6            30.63
================================================
========== 实验七结束 ==========
========== 实验七：运用 SIMULINK 对系统仿真 ==========
==================== 实验结果 ====================
输入幅值     超调量(%)
----------------------------------------
1            36.14
2            34.55
3            31.16
4            29.49
5            29.63
6            30.63
================================================
========== 实验七结束 ==========
```

4.4 图形输出

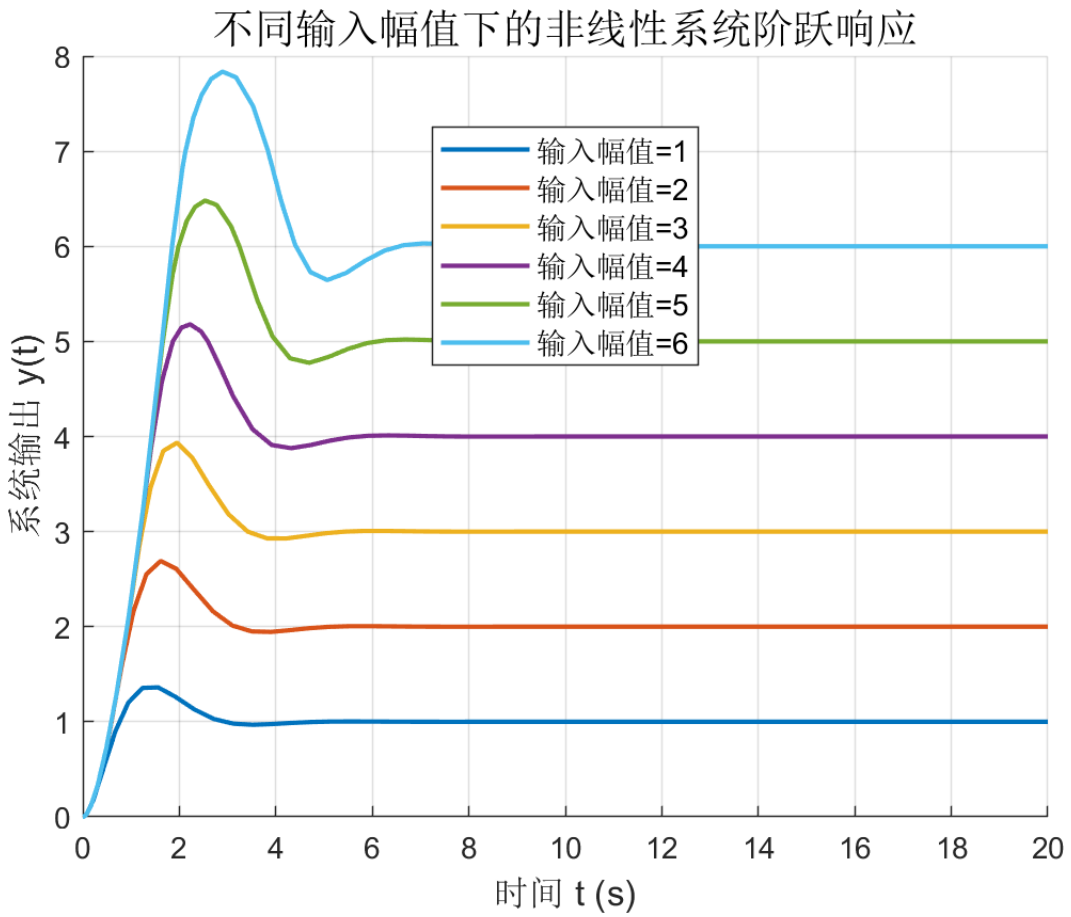


图 6-1 不同输入幅值（1~6）下的非线性系统阶跃响应对比

结果分析：

(1) 超调量随输入幅值增大呈先减小后稳定的趋势：A=1 时为 36.14%，A=4 时降至 29.49%，之后稳定在约 30%；

(2) 饱和非线性在大信号时限制了控制信号幅值（<=1），有效抑制了过度超调；

(3) 这体现了非线性系统区别于线性系统的核心特征：响应特性依赖于输入信号幅值（线性系统不依赖）；

(4) 饱和非线性在工程中普遍存在（放大器饱和、执行器限幅等），合理利用可改善动态性能。

5、分析与总结

(1) SIMULINK 提供直观的图形化建模环境，适合搭建含非线性环节的复杂系统；

(2) M 脚本通过 sim()实现批量参数化仿真，结合 for 循环可高效完成参数扫描；

(3) 饱和非线性使系统呈现输入幅值依赖的动态特性，大信号时控制受限、超调反而减小；

(4) for 循环+hold on+同图绘制是 MATLAB 参数扫描分析的标准方法，简洁高效。